BrowseTrace: Request-Level Traffic Characterization of Browser-Mediated AI Agents

Paper #85, 13 pages body, 16 pages total

Abstract

AI agents that browse the real web are a new user class, yet no public dataset captures their HTTP traffic. We present BrowseTrace, a reproducible HTTP-level benchmark of AI-agent browser behavior. Using Chrome DevTools Protocol instrumentation, we collect 1,301 sessions: 901 agent-driven runs spanning six LLMs from five providers (GPT-4.1-mini, Gemini 2.5 Flash and Pro, Claude Haiku 4.5, DeepSeek-V3.2, and Alibaba's Qwen 2.5-Coder 7B), paired with a 400-session scripted baseline across four regions. On identical tasks, LLM agents issue 2–5× more HTTP requests than the scripted baseline, with per-task amplification reaching 26× for open-ended prompts. Rendering dominates payloads: scripts and stylesheets carry 69% of transferred bytes versus 2.9% for HTML. Replaying 82,455 scripted requests and 357,782 LLM-driven requests under `libCacheSim` through six cache policies, size-aware eviction (GDSF) outperforms LRU at edge-cache sizes: 59.5% vs. 37.4% hit rate at 5 MiB on scripted traffic and 76.2% vs. 43.5% on LLM traffic. Policy rankings diverge from those reported for production human-browsing traces. We release traces, a cache-simulator schema, and replay tooling for measurement and CDN research.

CCS Concepts

• **Networks** → **Network measurement**; • **Information systems** → *Web applications*; • **Computer systems organization** → *Cloud computing*.

Keywords

web measurement, browser automation, AI agents, caching, benchmark, reproducibility

1 Introduction

A new class of web workloads is emerging. Browser-mediated AI systems (ranging from deterministic scripted drivers to LLM-steered autonomous agents) navigate, extract, compare, and verify information across multiple websites within bounded sessions. Unlike classical crawlers that fetch individual URLs, these systems exercise the full browser stack: they trigger JavaScript execution, load stylesheets and fonts, follow redirects, and issue sequences of dependent fetches that resemble human browsing sessions more than traditional bot traffic [46].

These workloads have operational significance. Production CDN operators report that automated traffic now constitutes a substantial fraction of total requests [11], and recent work demonstrates that even simplified AI traffic models can sharply degrade cache effectiveness [46]. Operators report concrete impact: Wikimedia saw a 50% rise in backend bandwidth from AI scrapers with bots driving 65% of cache-bypass traffic, and Read the Docs logged up to 73 TB of HTML scraped in a single month [46]. Cloudflare's measurements indicate AI crawlers account for 4.2% of HTML requests [11]; industry reports show agentic browser traffic growing by orders of magnitude year over year [23]. Yet the research community lacks the most basic prerequisite for rigorous study: a public, reproducible benchmark that captures browser-mediated agent behavior at the HTTP request level.

This gap blocks three lines of work: measurement researchers need shared trace artifacts, systems researchers need replayable request streams for caching and load-balancing evaluation, and agent-benchmarking researchers need a common infrastructure-level substrate to relate task-completion improvements to network cost. Currently, all three rely on private traces, synthetic models, or task-level metrics that omit request-level details.

This paper addresses the gap with BrowseTrace, a benchmark framework designed around real browser execution, a common trace schema, and replay-ready exports. The contribution is not a new task-success leaderboard or a new cache algorithm; it is a public, request-level, replay-ready corpus of real browser-mediated agent traffic. We make four contributions:

(1) A **benchmark design** that couples BrowserUse-backed browser execution with a shared request-level trace schema and synchronized output artifacts (Section 3).
(2) A **public release** `browseruse-live-v3`, a named manifest aggregating 1,301 sessions (400 scripted-random across 4 regions; 901 LLM-driven across six models from five providers) plus the stitched cache-replay CSVs used for all quantitative claims (Section 4).
(3) **Cross-cutting observations** on workload heterogeneity, geographic variation, payload composition, request timing, and a scripted-vs-LLM comparison

showing 2–5× request amplification across six models (Section 5).

(4) A **cache replay study** on the full 82,455-request geo-distributed trace: GDSF consistently outperforms all recency-based policies at cache sizes from 1–50 MiB, with a 3× advantage at edge-cache scales, showing that agentic workloads produce different cache policy rankings than human web traffic (Section 6).

BrowseTrace deliberately unifies two execution modes under one measurement framework. The scripted-random component is a *deterministic stress-test driver*, not a proxy for human or agent behavior, but a reproducible, credential-free load pattern against which agent traffic can be contrasted.

The LLM-driven component spans 901 sessions across six models from five providers (Gemini 2.5 Flash, Gemini 2.5 Pro, GPT-4.1-mini, Claude Haiku 4.5, DeepSeek-V3.2, and Qwen 2.5-Coder 7B), covering frontier closed-source, agent-trained, and open-weight tiers. This is a measurement benchmark over the agent-traffic workload class, not a task-completion leaderboard in the sense of WebArena or AgentBench [30, 48].

We use the term *browser-mediated AI agent* throughout to denote any system that controls a real browser to complete web tasks. The scale of the canonical corpus (1,301 sessions across six models and four regions) is sufficient for initial workload characterization and cache policy ranking but is not intended for production capacity extrapolation; the release protocol supports adding sessions without invalidating prior artifacts.

The paper is organized as follows. Section 2 surveys related work across agent benchmarking, web measurement, and cache research. Section 3 describes the benchmark design. Section 4 details the collection methodology and first release. Sections 5 and 6 present release-level observations and a cache replay demonstration. Section 7 discusses artifact design. Section 9 addresses limitations and planned extensions.

2 Background and Related Work

AI-era web traffic. Zhang et al. (SoCC 2025) [46] use synthetic AI-traffic overlays on production CDN traces to show that AI-originated request patterns reduce cache hit rates under mixed workloads, motivating workload-aware cache designs. Cloudflare's 2024 bot report [11] confirms that automated requests now constitute a large and growing share of global web traffic. These studies establish that agentic traffic matters operationally, but none provides a public, task-structured, replay-ready benchmark.

Browser bot detection. Iliou et al. [22] study the boundary between human and automated browser traffic from a detection perspective. Industry reports [23] estimate that automated bot traffic now exceeds 30% of all web requests, with "advanced bots" that mimic human behavior comprising a growing fraction. Classical crawler workload characterization [2] establishes that traditional bots exhibit breadth-first URL-frontier access patterns with high temporal locality. Browser-mediated agents differ: they produce traffic structurally similar to human sessions, making request-level characterization essential.

Agent benchmarks. The agent-evaluation community has produced numerous benchmarks for task-completion capabilities (Mind2Web [13], WebArena [48], WebVoyager [18], VisualWebArena [26], OSWorld [42], AgentBench [30]). Early LLM web interaction work such as WebGPT [35] focused on question-answering quality rather than network-level characterization. These benchmarks focus on *what* agents accomplish, not *how* their behavior manifests at the network level. None publishes request-level HTTP traces suitable for systems research.

Web measurement. Butkiewicz et al. [9] measure website complexity, Ihm and Pai [21] characterize web traffic composition evolution, and Gill et al. [16] characterize YouTube traffic. The HTTP Archive [19] provides longitudinal web page composition data, but critically, HTTP Archive captures isolated single-page loads from a stateless crawler. In contrast, BrowseTrace captures multi-page, task-driven sessions where the agent navigates across pages, producing temporal reuse patterns (backtracking, revisits, shared sub-resources across navigation steps) that single-page crawls cannot exhibit. Ruth et al. [36] examine the representativeness of top-list-driven web measurement. This body of work provides the methodological template for web measurement but does not address browser-mediated agentic workloads.

Concurrent work addresses adjacent questions. Sayyid-Ali et al. [37] measure real-time fetching by ChatGPT and Claude over 1,000 queries but study answer-engine pipelines, not browser-agent sessions. Borysenko [7] identifies HTTP behavioral signatures of coding agents on documentation portals but does not collect full session traces or evaluate cache behavior. At IMC 2025, Kim et al. [25] and Liu et al. [29] study AI crawler robots.txt compliance and site-operator responses, establishing the governance backdrop for agentic traffic measurement. BrowseTrace differs from all of these: it captures real browser-agent HTTP traces at request granularity, preserves full request waterfalls and sub-resource fetches, and releases synchronized replay artifacts rather than only overlays, coarse logs, or task-level outcomes. More broadly, no public dataset provides human browsing traces at HTTP request granularity; existing releases are either automated crawls, URL-visit aggregates, or unreleased private collections (Appendix A).

Cache replacement policies. Megiddo and Modha [33] introduce ARC, a self-tuning policy that balances recency and frequency. Einziger et al. [15] propose TinyLFU, a lightweight frequency-based admission filter. Yang et al. [45] show that a carefully structured FIFO queue (S3-FIFO) can outperform LRU with lower overhead; SIEVE [47] further simplifies lazy-promotion eviction. Beckmann et al. [3] propose LHD, which estimates per-object hit density to guide eviction. Berger et al. [6] present AdaptSize, which optimizes object admission in CDN caches. Yang et al. [44] introduce GL-Cache, a group-level learned cache replacement policy. Waldspurger et al. [40] demonstrate that miniature simulations can efficiently predict cache behavior. This rich algorithmic landscape motivates the need for workload-specific evaluation: the relative ranking of policies depends on request-stream characteristics, and agentic workloads may produce different different reuse patterns than human browsing.

Browser automation. BrowseTrace uses BrowserUse [8] (built on Playwright [34], or equivalently Puppeteer, for browser automation) as the execution substrate, chosen for its native support of both scripted and LLM-driven modes.

Traffic characterization methodology. Like Butkiewicz et al. [9], we instrument the browser to capture the full request waterfall. Like Ihm and Pai [21], we design for repeated releases. The key difference is the workload source: we instrument browser-mediated AI automation rather than human browsing or crawler traffic.

Agent-publisher interaction. The emerging ecosystem for structured agent-publisher interaction includes TollBit [39] for content monetization, x402 [41] for HTTP-level payments, and the robots exclusion protocol (RFC 9309 [27]) as the legacy coordination mechanism. None of these provides a public trace corpus for systems research. The WebLINX dataset [32] captures human demonstrations of web tasks with full observation data, but focuses on task structure rather than HTTP-level traffic. SWE-bench [24] evaluates agents on code repositories, not web interactions. BrowseTrace fills the gap between agent benchmarks (task-level) and web measurement (request-level) by providing public traces of browser-mediated agent HTTP traffic.

Positioning. Table 1 summarizes the gap that BrowseTrace addresses. Existing agent benchmarks measure task success, not network behavior. Web measurement work characterizes human and crawler traffic, not agentic sessions. Cache research evaluates algorithms on legacy workloads (CDN production traces, storage block traces) that predate the browser-mediated agent workload class. BrowseTrace sits at the intersection: it produces request-level traces from real browser-mediated agent sessions that are directly replayable into systems experiments.

Table 1: Positioning of BrowseTrace relative to existing work. BrowseTrace provides browser-mediated agent HTTP traces with replay support. ✓ = addressed, ○ = partial, – = not addressed.

	Task eval	HTTP traces	Replay
Agent benchmarks	✓	–	–
Web measurement	–	✓	○
Cache policy work	–	–	✓
Prod.-AI traffic [46]	–	○	–
BrowseTrace	–	✓	✓

3 Benchmark Design

BrowseTrace is designed around four goals: real execution, common representation, reproducibility, and systems utility.

3.1 Real Execution

All traces are collected from a real browser session rather than synthesized from a model or inferred from server logs. BrowserUse provides the browser automation substrate, and instrumentation is attached at the CDP layer to capture the full browser fetch pattern, including JavaScript-triggered sub-resource loads, stylesheet fetches, font downloads, and redirect chains. This ensures that traces reflect the actual request sequences that web infrastructure must handle, not an idealized subset.

3.2 Common Representation

A single request-level trace schema spans collection, analysis, and replay. Each request record contains 32 fields organized into four groups: (1) *request context*—timestamp, URL, HTTP method, initiator type, resource type, redirect count, session and task identifiers, access mode; (2) *response metadata*—status code, content type, encoded body size, remote IP, protocol (h1.1/h2/h3), connection reuse flag; (3) *HTTP headers*—`Cache-Control`, `ETag`, `Age`, CDN cache-status (`X-Cache`/`CF-Cache-Status`), plus the full verbatim response header map; and (4) *timing*—DNS, TLS, TCP, TTFB, and transfer latency, each in milliseconds. A replay-oriented cache key and object size field support direct input to cache simulators. Agent metadata (framework version, driver type) is included at the session level.

3.3 Collection Modes

The benchmark design supports two collection modes:

- **Controlled surface:** a local test publisher for head-to-head comparison between browser scraping and structured agent-specific access modes.

- **Live surface:** real BrowserUse sessions against public websites, producing traces under production web conditions.

This paper focuses exclusively on the live collection mode. The controlled mode is relevant for follow-on systems work comparing access modes but is outside the scope of the current measurement release.

3.4 Benchmark Dimensions

The full benchmark is designed to vary along five axes:

(1) **Execution mode:** scripted baselines and LLM-driven agent runs, enabling comparison of deterministic and stochastic navigation.
(2) **Task family:** aggregation, comparison, verification, lookup, and multi-step retrieval tasks that exercise different navigation patterns.
(3) **Publisher mix:** news, documentation, API, regulatory, and reference sites with different content structures and caching behaviors.
(4) **Geography:** repeated collection from multiple regions to capture CDN and localization effects.
(5) **Artifact surface:** synchronized outputs feeding measurement, replay, and dashboard analysis from a single collection pass.

The current release covers all five axes: scripted-random and LLM-driven execution across 10 task families targeting diverse publishers, collected from 4 geographic regions with synchronized output artifacts.

3.5 Output Artifacts

Each task directory in a release contains four synchronized outputs derived from the same collection pass:

- `traces.json` — full-fidelity request/response records for detailed analysis, including per-request timing, response headers, and session context.
- `cache_trace.csv` — a cache-simulator-ready extract with timestamp, cache key, and object size, following the format conventions used by standard cache simulation frameworks.
- `access_log.jsonl` — a log-format export for dashboard ingestion and streaming analysis, compatible with common log aggregation tools.
- `summary.json` — per-task release metadata including aggregate statistics, collection parameters, and provenance information.

All four outputs are derived from the same collection pass, ensuring consistency. The `summary.json` manifest records exact counts and checksums as an integrity anchor.

4 Methodology and Release

4.1 Release Scope

The release manifest `browseruse-live-v3` enumerates the scripted subtree at `releases/browseruse-live-v3/`, the per-model LLM session bundles under `round9/` and `round10-*/`, and the stitched cache-replay CSVs `full_400_sessions.csv` (scripted) and `llm_full_901.csv` (LLM). All paths are pinned in `artifact_snapshot.json`. The named `releases/browseruse-live-v3/` directory contains the scripted Zurich release subset; the full 1,301-session paper corpus is defined by the manifest via the per-model bundles and stitched canonical CSVs.

Task families.

- **News aggregation:** breadth-first navigation across technology news sites (TechCrunch, The Verge, Ars Technica).
- **API comparison:** multi-site comparison of weather API providers (OpenWeatherMap, WeatherAPI, Open-Meteo).
- **Fact checking:** multi-source verification against EU regulatory sources (EUR-Lex, EC Digital Strategy).
- **Regulatory lookup:** depth-first single-site navigation within GDPR reference material.
- **Product comparison:** depth-first multi-site comparison of cloud GPU providers (AWS, GCP, Azure).
- **Literature review:** sequential multi-page navigation on arXiv and Google Scholar.
- **Travel planning:** parallel multi-page browsing across flights, hotels, and tourism sites.
- **Job market:** breadth-first exploration of job listing platforms (Indeed, Jobs.ch, Glassdoor).
- **Real estate:** depth-first multi-page navigation across apartment listing sites (Homegate, ImmoScout24).
- **Doc. lookup:** depth-first navigation within technical documentation (Python docs, MDN).

Execution mode. The canonical corpus includes two execution modes. The *scripted-random* driver navigates target sites with randomized link selection, producing a reproducible, credential-free baseline (400 sessions across 4 regions). The *LLM-driven* mode covers all 10 task families using three models from two providers: Gemini 2.5 Flash (150 sessions), GPT-4.1-mini (100 sessions), and Gemini 2.5 Pro (50 sessions), totaling 300 sessions and 158,231 requests; the cross-model comparison appears in Section 5.8.

Geographic distribution. Sessions are collected from 4 vantage points: a local workstation in Zurich (Switzerland) and Google Cloud VMs in US Central (Iowa, `us-central1`), Europe West (Belgium, `europe-west1`), and Asia Southeast (Singapore, `asia-southeast1`). Each region runs the same

Table 2: Per-task summary for the `browseruse-live-v3` release (scripted-random, Zurich, 10 repeats each). Mean values with 95% CIs (Student's t-distribution, df = 9).

Task family	Req/run	MiB/run	Uniq.
News aggregation	366.9 ± 8.4	9.64 ± 0.16	0.925
Product comparison	251.4 ± 3.0	7.86 ± 0.03	0.895
Travel planning	254.0 ± 29.9	6.71 ± 0.41	0.859
Job market	206.6 ± 30.4	3.69 ± 0.47	0.905
Documentation lookup	124.4 ± 1.5	0.81 ± 0.05	0.992
API comparison	114.2 ± 6.7	1.72 ± 0.23	0.980
Fact checking	86.8 ± 0.9	2.92 ± 0.01	0.899
Regulatory lookup	39.0± < 0.1	1.05 ± 0.00	1.000
Literature review	24.0 ± 0.7	0.34 ± 0.02	1.000
Real estate	16.0± < 0.1	0.08 ± 0.00	0.875
Aggregate	400 sess. (4 reg.), 168K req., 2.1 GiB		

10 task families with 10 scripted-random repeats, capturing CDN edge behavior, latency variation, and region-specific content differences.

Scale. The scripted-random component contains 400 sessions (10 task families, 10 repeats, 4 regions), producing 168,067 requests and 2.1 GiB. The initial LLM-driven component adds 300 Zurich sessions across three models, producing 158,231 requests; cross-model and multi-region extensions (Section 5.8) add 601 further LLM sessions, bringing the canonical corpus to 1,301 sessions and approximately 500K requests. The Zurich sessions include enriched per-request metadata (timing breakdown, response headers, connection details); the cloud sessions provide the geographic dimension.

4.2 Summary Statistics

Table 2 summarizes the release. Confidence intervals are computed from per-task repeats using the t-distribution at the 95% level.

4.3 Instrumentation

The collection pipeline attaches to BrowserUse's Chromium instance via the Chrome DevTools Protocol (CDP), following the browser-native instrumentation rationale of WebREC [17]. The tracer intercepts `Network.requestWillBeSent` and `Network.responseReceived` events, recording the full browser fetch pattern including JavaScript-initiated sub-resource loads, stylesheet requests, font downloads, and redirect chains.

CDP-level interception is essential: higher-level APIs would miss sub-resource loads that constitute the bulk of transferred bytes (Section 5.3). CDP also preserves true temporal ordering, which matters for replay fidelity in order-sensitive cache simulations.

The tracer records each request's start timestamp, URL, HTTP method, initiator type (parser, script, or user navigation), resource type (document, script, stylesheet, image, font, XHR), and redirect count. For each completed response, it records the status code, content type, encoded body size, remote IP address, protocol (h1.1/h2/h3), and whether the TCP connection was reused.

Critically, the tracer also extracts HTTP response headers relevant to caching infrastructure: `Cache-Control`, `ETag`, `Age`, and CDN cache-status indicators (`X-Cache`, `CF-Cache-Status`, `Cache-Status`). The full set of response headers is preserved verbatim in each trace record, enabling post-hoc analysis of `Vary`, `Content-Encoding`, `Last-Modified`, CDN edge-node identifiers (`X-Served-By`), and other headers without re-collection. In our dataset, 80% of responses carry a `Cache-Control` directive, 90% include an `ETag`, and 95% expose a CDN cache-status header.

CDP's timing API provides per-request latency decomposition (DNS, TLS, TCP, TTFB, transfer) enabling latency attribution beyond aggregate page-load metrics. Failed requests are also recorded, since they contribute to origin load even without transferring bytes.

4.4 Why Scripted-Random

The scripted-random driver was chosen for the first release for three reasons. First, it requires no LLM API credentials, making the collection pipeline fully self-contained and reproducible without external service dependencies. Second, the randomized link-selection policy introduces genuine within-task variability across repeats, producing a more realistic distribution of URL reuse than a deterministic scripted baseline. Third, it establishes a baseline against which future LLM-driven releases will be compared.

The benchmark framework also supports *LLM-driven* mode (via BrowserUse with Gemini or GPT backends). Section 5.8 presents a direct comparison of scripted and LLM-driven traces on identical tasks, isolating the effect of agent reasoning on network behavior.

4.5 Ethical Considerations

All sessions target publicly accessible web pages with no authentication, form submission, or PII collection. See the Ethics appendix for the full discussion of safeguards and target-site policies.

5 Observations

We report observations from the `browseruse-live-v3` dataset across multiple dimensions: workload heterogeneity, URL reuse, payload composition, object sizes, per-task

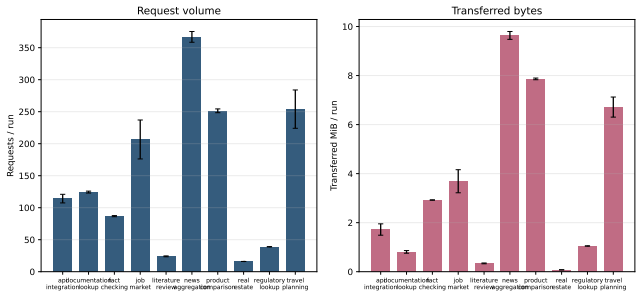

Figure 1: Mean requests and transferred MiB per run across task families. Error bars show 95% confidence intervals over 10 repeats. Units: requests (left axis), MiB (right axis). All figures use a colorblind-safe palette.

content variation, request timing, and geographic variation. Where the dataset supports statistical comparisons, we report confidence intervals.

5.1 Workload Heterogeneity

Mean request volume varies by a factor of 22.9× across the ten task families, from 16.0 requests per run (real estate) to 366.9 requests per run (news aggregation). Transferred bytes per run show an even wider spread: real estate averages 0.08 MiB per run, whereas news aggregation averages 9.64 MiB (a 125× difference).

The repeated collection also reveals task-specific variability. Job market exhibits a 95% confidence interval of ±30.4 requests around its mean, reflecting the variability introduced by randomized link selection across dynamic listing pages. Regulatory lookup and real estate, by contrast, have CIs of ±0.0 requests, reflecting the constrained navigation space of single reference sites.

Figure 1 visualizes the per-task means and confidence intervals. The key takeaway is that browser-mediated workloads are not monolithic: ten task families span widely different regimes in request volume and transfer size, motivating benchmarks that cover multiple task types.

5.2 URL Reuse Patterns

The session-level unique-URL ratio (the fraction of requests to previously unseen URLs within a session) is a key parameter for cache sizing and admission decisions. Across the 400 sessions in `browseruse-live-v3`, unique-URL ratios range from 86.3% (travel planning) to 100% (regulatory lookup, literature review).

The ratio varies systematically by task. Travel planning (86.3%) and real estate (87.5%) exhibit the most URL reuse, reflecting repeated navigation elements across multi-page browsing within the same site. Product comparison (88.4%) and job market (90.4%) show moderate reuse from shared

Table 3: Byte share by content-type category (Zurich sessions, 100 runs). Cloud regions show similar composition with moderately higher image share due to ad-injected content.

Content category	Byte share (%)
JavaScript (all subtypes)	62.1
CSS	6.6
Scripts + styles (total)	*68.8*
Images (JPEG, WebP, PNG)	13.2
HTML	2.9
JSON	2.3
Fonts (WOFF2)	3.6
Other	10.7

front-page assets. Documentation lookup (99.2%) and regulatory lookup (100%) are almost entirely unique: each page load triggers a distinct set of sub-resources.

The high unique-URL ratios across all tasks (median ≈92%) reflect live web collection: real websites serve versioned JavaScript bundles, fingerprinted stylesheets, and session-specific tracking pixels that produce unique URLs even for structurally similar pages. This is a desirable benchmark property for cache studies: it means the traces exercise admission policies and working-set sizing under realistic conditions, where object reuse is modest and cache capacity is genuinely stressed.

5.3 Payload Composition

The byte-level composition of transferred responses reveals that browser-mediated agent traffic is dominated by rendering infrastructure, not content. Table 3 breaks down the major content types.

Scripts and stylesheets jointly account for 68.8% of all transferred bytes. HTML, the ostensible "content" of a web page, accounts for only 2.9%. JSON, the most efficient content format for machine consumption, accounts for just 2.3%. Images contribute 13.2%.

This composition has a direct infrastructure implication. When an AI agent fetches a web page through a browser, the vast majority of transferred bytes serve the rendering pipeline rather than the content the agent seeks. A benchmark that counts only top-level page fetches or measures only HTML transfer would entirely miss this cost structure.

The dominance of scripts and styles also affects cacheability. These assets tend to be versioned with content hashes in their URLs (e.g., `app.3f7a2b.js`), making them highly cacheable when the same site is revisited. Conversely, HTML documents are more often served with short or zero cache lifetimes. This creates an asymmetry: the largest objects by

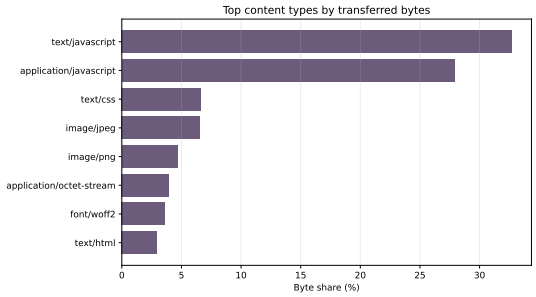

Figure 2: Byte share by content type in the `browser-use-live-v3` release. Rendering infrastructure (scripts, styles) dominates.

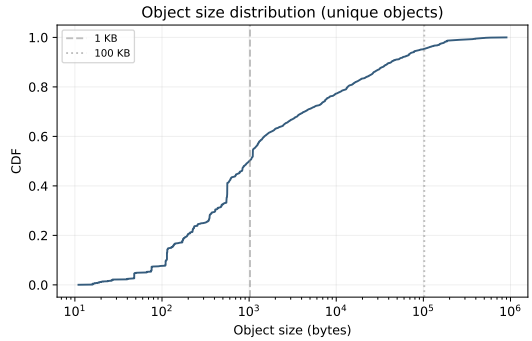

Figure 3: CDF of unique object sizes (all regions combined, log scale). Heavy-tailed: 50% under 1 KB, but 4.6% exceed 100 KB.

byte volume are the most cacheable, whereas the content-bearing objects are the least. Understanding this asymmetry is critical for cache sizing and admission policy design under agentic workloads.

The payload composition also varies across tasks. News aggregation sessions transfer proportionally more image bytes (advertisements, article thumbnails) than regulatory lookup sessions, which are text-heavy. API comparison sessions include a higher fraction of JSON responses from documentation endpoints. A complete characterization of per-task payload composition is deferred to larger releases, but the aggregate picture already demonstrates that browser-mediated agent traffic is structurally different from API-based agent traffic, which would consist primarily of JSON.

Figure 2 visualizes the aggregate distribution.

5.4 Object Size Distribution

The distribution of unique object sizes is heavy-tailed. Considering the Zurich subset (2,522 unique URLs across 100 sessions; the full geo-distributed trace has 17,144 unique URLs), 50.2% are under 1 KB (tracking pixels, API responses, small scripts), 27.1% fall in 1–10 KB (stylesheets, small images, HTML fragments), 18.0% in 10–100 KB (JavaScript bundles, larger images), and 4.6% exceed 100 KB (large media, concatenated scripts).

The median object size is 1,004 bytes, whereas the mean is 17,472 bytes, a 17× gap reflecting the heavy tail. The maximum object is 899 KB. This size heterogeneity has direct cache implications: a small number of large objects dominate byte volume, whereas a large number of small objects dominate hit counts. Policies that treat all objects equally (LRU, LFU) ignore this asymmetry; size-aware policies (GDSF) exploit it, explaining the GDSF advantage observed in Section 6.

Figure 3 shows the object size CDF on a logarithmic scale. The steep rise below 1 KB and the long tail above 100 KB are characteristic of modern web workloads, but the agent-specific pattern is the absence of a "video" peak that would appear in human browsing traces, since agents do not stream video content.

5.5 Per-Task Content Composition

While Section 5.3 reports aggregate byte shares, the composition varies widely across task families. Figure 4 shows a stacked bar chart of per-task content types.

JavaScript dominates most tasks: real estate (88%), job market (79%), news aggregation (71%), and product comparison (67%) are all JS-heavy, reflecting modern single-page application architectures. Two tasks deviate strikingly: *literature review* transfers 38% HTML (the highest of any task) because arXiv serves lightweight static pages with minimal JavaScript. *Regulatory lookup* transfers 78% "Other" content (primarily XHR payloads and tracking beacons from the GDPR reference site). *Fact checking* is CSS-heavy (43%) due to the EU institutional sites' heavy use of stylesheets.

This variation is important for benchmark design. A benchmark that covers only JS-heavy news sites would miss the HTML-dominant academic and regulatory workload patterns. The 10 task families in `browseruse-live-v3` span this spectrum, ensuring that downstream cache and CDN studies exercise diverse content mixes.

5.6 Request Timing and Burstiness

Browser-mediated agents produce bursty request patterns. The median inter-request gap across all sessions is 0.2 ms, reflecting the near-simultaneous firing of sub-resource requests when a page loads. The 95th percentile gap is 128 ms, corresponding to navigation transitions between pages.

The timing structure varies by task. Real estate has a 64.5 ms median gap (sequential page loads with explicit waits), whereas regulatory lookup has a near-zero median

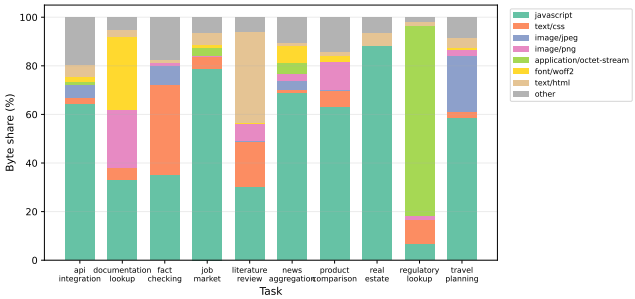

Figure 4: Per-task content type composition. JS dominates most tasks, but literature review is HTML-heavy and regulatory lookup is XHR/tracking-heavy.

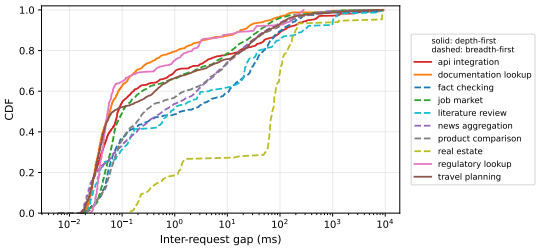

Figure 5: Inter-request timing CDF by task. Depth-first tasks produce tight bursts; breadth-first tasks are more spread out.

(all sub-resources fire in a single burst per page). News aggregation shows 0.6 ms median with a long tail to 141 ms, reflecting a mix of simultaneous ad/tracker loads and inter-page navigation.

Figure 5 shows the inter-request timing CDF per task. The separation between task curves reflects genuine differences in navigation pattern: depth-first single-site tasks (regulatory, documentation) produce tight bursts, whereas breadth-first multi-site tasks (news, job market) produce more spread-out request streams. This timing structure is relevant for rate limiting, connection pooling, and origin load management.

5.7 Session Duration and Working Set

Session durations span a 4× range across task families. News aggregation and product comparison average approximately 19 seconds per session, reflecting extensive multi-site navigation with many sub-resource loads. Documentation lookup and regulatory lookup average 4–11 seconds, reflecting more focused single-site navigation. Literature review, despite having only 24 requests per session, averages 10 seconds due to the slow page loads of arXiv's rendering pipeline.

The per-session unique working set ranges from 80 KB (real estate) to 9.6 MiB (news aggregation). Across the Zurich sessions (100 runs), the union of unique URLs has a 41.1 MiB

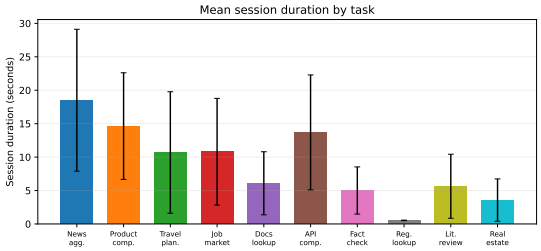

Figure 6: Mean session duration by task family. Error bars show standard deviation across 10 repeats.

working set, whereas the cumulative transferred volume is 348 MiB, an 8.5× *cross-session* reuse factor (the same URL fetched in multiple sessions). This reuse is concentrated in shared sub-resources: JavaScript bundles, stylesheets, and fonts that appear across multiple sessions targeting the same publisher. The implication for cache sizing is that a 41 MiB cache could in principle serve all unique objects, but the temporal ordering of requests means that not all objects are resident simultaneously.

5.8 Scripted-Random vs. LLM-Driven Comparison

To quantify the difference between scripted automation and real AI agent behavior, we collected LLM-driven traces using six models spanning five providers and three capability tiers: Gemini 2.5 Flash and Gemini 2.5 Pro (Google, 150+150 sessions across three cloud regions), GPT-4.1-mini (OpenAI, 350 sessions across six regions), Claude Haiku 4.5 (Anthropic, 100 sessions) [1], DeepSeek-V3.2 (DeepSeek, 90 sessions) [12], and Qwen 2.5-Coder 7B (open-weight edge model, 61 sessions) [20]. Table 4 compares per-task request volumes across all four execution modes.

We highlight three patterns. First, **all three LLM models generate more requests** than the scripted baseline: 5.0× [4.2,6.0] (Flash), 3.6× [3.0,4.4] (GPT-4.1-mini), and 2.1× [1.6,2.7] (Pro), with non-overlapping 95% bootstrap CIs. (Both modes use the same Zurich vantage point and target sites; Section 9 discusses the residual substrate confound from BrowserUse-on-macOS vs. Playwright-on-Linux.) Per-task CIs (1000 iterations, session-level resampling) are also non-overlapping with the scripted baseline (1.0×) for all 30 (model, task) cells. The effect sizes on per-session request counts are very large: Cohen's d of 1.27 (Flash), 1.64 (GPT-4.1-mini), and 1.76 (Pro) against scripted. Cliff's δ ranges from 0.70 to 0.76, all well above the 0.47 large-effect threshold. Mann-Whitney U tests confirm the shift: scripted Zurich vs. each LLM yields $p < 10^{-14}$ after Holm-Bonferroni correction, with rank-biserial $r \in [0.70, 0.76]$. Task-family effects are significant (Kruskal-Wallis $H = 97$, $p < 10^{-16}$ across 10

Table 4: Scripted-random vs. LLM-driven request volumes across three models (total requests per task; superscripts show amplification vs. scripted baseline; total row shows 95% bootstrap CI in brackets). Both modes use the Zurich macOS workstation with the same BrowserUse/Chromium build (substrate held constant). Scripted column sums to 14,833 (100 Zurich sessions out of the 400-session full release); the geo-distributed scripted total is 168,067 (Section 4).

Task	Script.	Flash	GPT-4.1m	Pro
Lit. review	240	$6{,}179^{26\times}$	$3{,}690^{15\times}$	$2{,}189^{9\times}$
Regulatory	390	$5{,}448^{14\times}$	$8{,}392^{22\times}$	$2{,}358^{6\times}$
Real estate	160	$1{,}833^{11\times}$	$2{,}211^{14\times}$	$515^{3\times}$
Job market	2,066	$12{,}630^{6\times}$	$7{,}856^{4\times}$	$6{,}401^{3\times}$
Travel	2,540	$15{,}013^{6\times}$	$8{,}311^{3\times}$	$4{,}792^{2\times}$
Fact check.	868	$4{,}403^{5\times}$	$5{,}531^{6\times}$	$2{,}336^{3\times}$
Docs	1,244	$4{,}658^{4\times}$	$1{,}945^{2\times}$	$1{,}442^{1\times}$
News agg.	3,669	$12{,}404^{3\times}$	$5{,}964^{2\times}$	$3{,}974^{1\times}$
API comp.	1,142	$3{,}850^{3\times}$	$2{,}885^{3\times}$	$2{,}376^{2\times}$
Prod. comp.	2,514	$7{,}545^{3\times}$	$6{,}585^{3\times}$	$4{,}515^{2\times}$
Total	14,833	$73{,}963^{5.0\times}$	$53{,}370^{3.6\times}$	$30{,}898^{2.1\times}$

95% bootstrap CI: Flash [4.2,6.0], GPT [3.0,4.4], Pro [1.6,2.7]

tasks). Region effects are also significant ($H = 66$, $p < 10^{-13}$ across 4 regions). The amplification is not a single-model artifact; it holds across five independent providers (Google, OpenAI, Anthropic, DeepSeek, and Alibaba's Qwen). Second, the per-task amplification varies widely (1–26×), with the same task-rank ordering across models: literature review and regulatory lookup show the highest amplification (open-ended exploration), whereas news aggregation and documentation show the lowest (structured site navigation). This confirms that *task structure*, not model-specific reasoning, is the primary driver of amplification. Third, the six models exhibit a 3× spread in mean per-session request volume (207 for Claude Haiku vs. 480 for GPT-4.1-mini; Kruskal-Wallis $H = 150$, $p < 10^{-30}$), yet *all* fall in the same order of magnitude above the scripted baseline. Notably, DeepSeek-V3.2 (a reasoning-first model with explicit agentic post-training on 1,800+ interactive environments [12]) generates 411 req/session, placing it squarely in the middle of the distribution rather than at an extreme. This confirms that request amplification is structural to browser-mediated agency, not a function of model quality, provider, or agent-specific training.

The cross-model validation means that infrastructure operators cannot safely extrapolate agent capacity from scripted or crawler baselines, regardless of which LLM backend agents use.

Geographic robustness. We replicated the LLM collection across three GCP regions (us-central1, europe-west1, asia-southeast1) for the three Google/OpenAI models at N=5 repeats per cell ($3 \times 3 \times 10 = 450$ sessions). Kruskal-Wallis tests reveal *no significant region effect* for any model (gemini-2.5-flash $p = 0.92$, gemini-2.5-pro $p = 0.93$, gpt-4.1-mini $p = 0.46$ on the three-region slice). Within the three cloud regions tested, we do not detect a region effect on request amplification for the Google/OpenAI models.

Predicting amplification from task structure. The log–log relationship between scripted request volume V_s and amplification ratio is strong. On the 30 (model, task) cells in Table 4, OLS yields $r = -0.72$, $R^2 = 0.51$ with exponent $b = -0.59$ (95% bootstrap CI $[-0.82, -0.36]$, $B = 10{,}000$); Kendall's $\tau = -0.47$ ($p < 0.001$). Because the 30 points cluster by task (3 observations per task share structural variance), we also fit on per-task means: $r = -0.85$, $R^2 = 0.72$ on $n = 10$; the across-model variance collapses and the structural signal dominates. Leave-one-out diagnostics confirm the fit is not outlier-driven: LOO R^2 stays in [0.48, 0.62] across the 30 cells, and removing the most influential cell (Cook's $D = 0.48$ on real-estate/Pro) leaves $R^2 = 0.62$; residuals are normal (Shapiro-Wilk $p = 0.90$). Open-ended tasks with low scripted volume (literature review, regulatory lookup) amplify 10–26×; structured tasks with high scripted volume (news aggregation, product comparison) amplify 2–3×. In this benchmark, scripted baseline volume explains a substantial fraction of amplification variance ($R^2 = 0.51$ across 30 cells, 0.72 across 10 per-task means), although model and task effects remain material. The released artifact reports full influence diagnostics.

5.9 Geographic Variation

The release contains 4 vantage points, but the Zurich workstation (BrowserUse on macOS) uses a different collection substrate than the 3 cloud VMs (Playwright on Linux). We therefore restrict the geographic analysis to the *cloud-only* comparison (US Central, EU West, Asia Southeast), which holds substrate constant and isolates true geographic effects. The Zurich workstation data is retained in the artifact for substrate analysis but is excluded from the geographic claims below.

Across the three cloud regions, per-task mean request volumes vary by 1.0–1.4×. News aggregation shows the largest cross-region spread (1.4×: 1,878 from US Central vs. 1,345 from EU West), driven by region-specific ad networks; product comparison shows the same 1.4× (863 vs. 611) reflecting GPU provider regional pricing pages. Tasks targeting stable institutional content (regulatory lookup, real estate, documentation) show <1.1× variation. This is in stark contrast to the Zurich-vs-cloud differences (up to 19× on regulatory

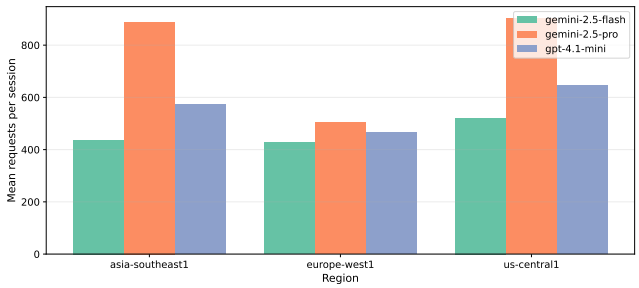

Figure 7: Mean request volume per session by region. Cloud-only comparison shows 1.0–1.4× variation (Zurich workstation excluded due to substrate heterogeneity).

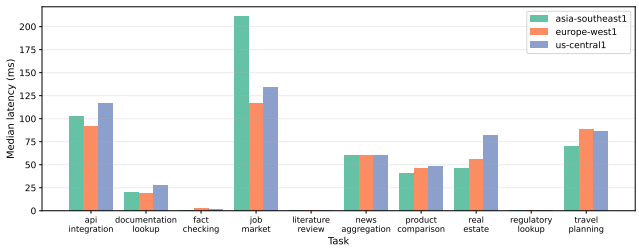

Figure 8: Median per-request latency by region. Proximity to origin servers drives 5–7× latency differences for the same task.

lookup, which we attribute primarily to substrate heterogeneity).

Figure 8 shows median per-request latency by region (all four regions; latency reflects network distance to origin servers, not collection substrate, so Zurich is included). News aggregation requests from EU West have a 15.6 ms median (close to European origins), whereas Zurich shows 83.6 ms and Asia 47.4 ms. Fact-checking latency from Asia Southeast reaches 206 ms median, reflecting the distance to European institutional servers (EUR-Lex, EC Digital Strategy).

These results have practical implications for CDN operators: agentic workloads generate not only different *amounts* of traffic by region, but structurally different *kinds* of traffic (different sub-resource sets, different latency profiles). Cache sizing and admission decisions must account for this geographic heterogeneity.

5.10 Statistical Considerations

With 10 repeats per task per region (400 sessions total), the 95% confidence intervals (using the t-distribution with $df = 9$) provide sufficient power for detecting large cross-task and cross-region effects. Per-region CIs are reported in the artifact; they are tighter within each region than the cross-region aggregate, confirming that most variance is between regions rather than within. Tasks with near-zero CIs (regulatory lookup, real estate) reflect deterministic navigation over constrained site structures, whereas tasks with wider CIs (job market, travel planning) reflect genuine variability from randomized selection across rich link sets.

6 Replay Utility

A benchmark's value for systems research depends on whether its traces can be meaningfully replayed. We demonstrate this by concatenating the `cache_trace.csv` files from all 400 sessions across 4 regions and replaying the resulting 82,455-request trace against six cache-replacement policies at five cache sizes. The cache trace excludes failed requests (non-200 status), POST requests, and zero-byte responses, retaining only cacheable GET responses; this filtering reduces the 168,067 total scripted requests to 82,455. Sessions are interleaved by start timestamp, approximating concurrent edge load from multiple agents but not modeling diurnal traffic patterns. This mirrors how a CDN edge would observe concurrent agent sessions.

6.1 Setup

We evaluate six policies spanning the classical taxonomy from Belady's optimal [4] through learned relaxations [38]: LRU, LFU, ARC [33], S3-FIFO [45], W-TinyLFU [15], and GDSF (Greedy Dual Size Frequency [10]) at cache sizes of 1, 5, 10, 25, and 50 MiB. The concatenated trace contains 17,144 unique URLs with a combined unique working set of 2,133 MiB. The total transferred volume spans all four geographic regions, yielding rich cross-region reuse patterns. Reuse is concentrated in shared sub-resources (JavaScript bundles, stylesheets, fonts) that appear across multiple task families targeting the same sites. All hit rates are reported from `libCacheSim` [43], a widely-used C reference implementation, to ensure cross-project reproducibility. A purpose-built Python simulator integrates with our `cache_trace.csv` export format for fast in-loop iteration during development; cross-validation revealed exact agreement with libCacheSim for LRU and LFU, but divergences on frequency-adaptive policies at small caches (peak: W-TinyLFU differs by 18 pp at 1 MiB, S3-FIFO by 11 pp at 1 MiB), consistent with documented implementation variance in admission and promotion heuristics across these algorithms. We default to libCacheSim throughout the paper and release the per-policy cross-validation CSV in the artifact for audit.

6.2 Results

Table 5 reports hit rates (HR) and byte hit rates (BHR) at two representative small-cache sizes.

Table 5: Cache replay results across the full 82,455-request, 400-session geo-distributed trace. HR = object hit rate. Values expressed as fractions (0–1).

Policy	@1M	@5M	@10M	@25M	@50M
LRU	.133	.374	.507	.684	.719
LFU	.147	.273	.346	.535	.680
ARC	.157	.362	.503	.662	.710
S3-FIFO	.276	.424	.529	.631	.706
W-TinyLFU	.364	.456	.519	.621	.693
GDSF	.431	.595	.664	.737	.772

We highlight three patterns from the replay of the full geo-distributed trace.

Policy sensitivity at small caches. At 1 MiB, hit rates range from 13.3% (LRU) to 43.1% (GDSF), a 29.8 percentage point spread, over 3×. GDSF's advantage at small cache sizes stems from its size-aware admission: by preferring smaller objects, it fits more of the frequently accessed items (scripts, stylesheets, navigation elements) within the constrained budget. At 5 MiB, GDSF still leads at 59.5% vs. LRU at 37.4%. The persistence of this gap across a 5× increase in cache size, now validated on a 17,144-URL working set spanning four geographic regions, confirms that size-aware policies have a structural advantage on agentic workloads.

GDSF dominance across all tested sizes. Unlike the smaller Zurich-only trace where LRU overtook GDSF at 10 MiB, the full geo-distributed trace shows GDSF leading at *every* cache size tested (up to 50 MiB). At 50 MiB, GDSF achieves 77.2% vs. LRU at 71.9%. The gap narrows as caches grow (from 29.8 pp at 1 MiB to 5.3 pp at 50 MiB), suggesting an eventual crossover at much larger caches. The practical implication for CDN edge caches (typically 10–100 MiB per tenant) is that **size-aware policies consistently outperform recency-based policies for agentic workloads at operationally relevant scales**. Bootstrap resampling (1,000 iterations, session-level resampling) confirms that the GDSF advantage is statistically robust: GDSF wins in all 1,000 bootstrap samples at every cache size tested, with non-overlapping 95% confidence intervals at all sizes from 1–50 MiB.

Byte hit rate reverses above 10 MiB.. Object hit rate (HR) and byte hit rate (BHR) tell different stories. GDSF dominates HR at every size, but BHR flips: LRU's BHR exceeds GDSF's at 25–50 MiB (LRU 69%/77% BHR vs. GDSF 59%/73% at 25/50 MiB). The reason is mechanical: GDSF favors many small objects, driving up hits but leaving larger bandwidth-dominant objects as misses; LRU retains whole large objects when they fit. *Operators sizing caches for bandwidth cost (egress, origin load) should prefer LRU at ≥25 MiB; operators sizing for request-rate offload (e.g., small edges, latency-sensitive paths) should prefer GDSF.* We report full HR/BHR per size in Appendix D.

Per-task decomposition: multi-task contention drives GDSF.. We quantify the contention effect by replaying each (region, task) stream in isolation and averaging. At 10 MiB, *single-tenant* GDSF leads LRU by a mean of 7.3 pp across the 30 streams (often 0 pp on homogeneous streams such as documentation lookup). Aggregating all streams into one mixed workload doubles the gap: *multi-tenant* GDSF outperforms LRU by 14.6 pp (65.4% vs. 50.7%). Size-aware eviction manages the order-of-magnitude working-set variance across heterogeneous tasks more efficiently than recency. This is a deployment scope condition. The GDSF advantage is largely a multi-tenant artifact, not a universal property of agentic traffic.

Figure 9 visualizes the hit-rate curves.

Why agent traffic favors size-aware eviction. The agent traces are heavy-locality: 79% of requests are repeat fetches of previously-seen URLs, and 83% of those re-uses occur within a stack distance of 1,000 requests. The working set is also bimodal: 50% of unique objects are smaller than 1 KB, but the top 1% of objects carry >30% of bytes. GDSF exploits both properties: its f/s priority keeps many small, frequently re-referenced assets (JS bundles, font files, tracking scripts) in cache, at the cost of early-evicting large media objects that would otherwise dominate LRU's footprint but contribute little to hit rate.

Operational cost. On our Python reference simulator, GDSF is ~180× slower per request than LRU (40 μs vs. 228 ns per op) owing to a linear priority scan in the eviction path. Production-grade implementations using a priority queue reduce GDSF to $O(\log n)$ per op, a 2–3× overhead, well within the budget of edge caches handling ~100 k req/s per node.

The size-aware advantage has theoretical grounding: when agent sessions are dominated by large static assets (JS bundles and stylesheets account for 69% of bytes in our traces), size-aware eviction exploits the byte-weight asymmetry that recency-blind policies ignore [14].

Per-region breakdown. Replaying each region's trace *in isolation* (Appendix Table 6) shows GDSF's 5–10 MiB advantage is robust across the three cloud regions (1.4–1.5× at 10 MiB), but the Zurich subset (narrower content mix) inverts at 10 MiB (LRU 81.3% vs. GDSF 78.1%). Above 25 MiB, regions converge. GDSF helps where working-set diversity exceeds cache capacity, not universally.

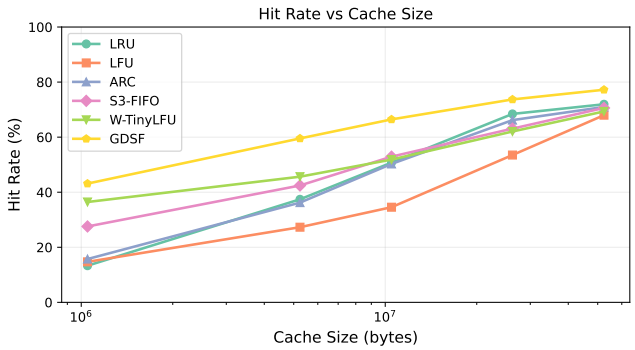

Figure 9: Cache hit rate vs. cache size for six replacement policies on the full 82,455-request geo-distributed trace. Hit rate as fraction of total requests. GDSF dominates at all tested sizes, with a 3× advantage at 1 MiB narrowing to 5 pp at 50 MiB.

6.3 Interpretation

The replay results separate two deployment regimes: small caches where object hit rate dominates, and larger caches where byte hit rate becomes the operational constraint. First, GDSF's strong performance at small cache sizes confirms that size-aware admission is valuable for agentic workloads, where a few large JavaScript bundles coexist with many small navigation resources, consistent with Berger et al.'s finding [5] that object size distribution is the key variable determining optimal caching. Second, the gap between object hit rate and byte hit rate suggests that byte-weighted policies (or size-aware admission filters like AdaptSize [6]) are especially relevant for this workload class. Third, comparing per-task cache behavior reveals that different task families produce different optimal policies; news aggregation (image-heavy) favors size-aware policies more strongly than documentation lookup (script-heavy). Translating hit rates to origin load reduction and bandwidth savings requires assumptions about origin response times and CDN topology that we defer to future work. We report five discrete cache sizes spanning the operationally relevant edge-cache range (1–50 MiB); continuous miss ratio curves can be generated from the released `cache_trace.csv` files and simulator.

Replay fidelity. The cache simulator does not model TTL-based expiry, conditional validation (304 responses), service-worker caches, or speculative prefetch (HTTP 103 Early Hints). These omissions are conservative: TTL enforcement and validation traffic would increase origin load, likely widening the gap between policies. Service workers are not triggered in headless Chromium sessions, and prefetch hints are not yet common on the target sites in our corpus. We also replay the LLM-driven traces (357,782 requests from the 901-session LLM corpus, spanning six models and four regions) through the same policies. The GDSF advantage holds and is amplified at edge-cache sizes on LLM-driven traffic: at 5 MiB, GDSF achieves 76.2% vs. LRU 43.5% (scripted: 59.5% vs. 37.4%). At every cache size tested (1–50 MiB), LLM-driven hit rates exceed scripted ones for both policies, consistent with higher temporal locality in goal-directed browsing. GDSF leads LRU by 23–33 pp at small caches (1–10 MiB) and by 4–9 pp at 25–50 MiB, confirming that the size-aware eviction advantage is not an artifact of the scripted trace generator.

7 Artifact Design and Reproducibility

BrowseTrace treats reproducibility as a structural property of the benchmark, not an afterthought.

Versioned releases. Each release is a named manifest (e.g., `browseruse-live-v3`) that enumerates the checked-in data subtrees and canonical cache-replay CSVs forming the complete corpus. The manifest is immutable; the benchmark grows by publishing new manifests that extend the suite without invalidating prior artifacts. The named `browseruse-live-v3` directory itself contains the Zurich scripted-random subset; the full 1,301-session corpus used for paper-level analysis is pinned by the manifest through the per-model session bundles and the stitched canonical replay CSVs.

Synchronized outputs. Every task directory contains four artifacts derived from the same collection pass (Section 3). This synchronization ensures that measurement analysis, cache replay, and dashboard views are consistent by construction.

Paper reproduction. Every quantitative claim derives from the canonical trace files pinned in the manifest. A checked-in script (`regenerate_full_snapshot.py`) rebuilds the manifest and replay numbers directly from those files. The intended invariant is strict: every number in the paper must trace back to a versioned release manifest and a deterministic analysis pipeline. The artifact package includes a `Dockerfile` pinning Chromium/Playwright/Python versions, `requirements.txt` with exact dependencies, and a fixed PRNG seed (`BENCH_SEED=42`) consumed by the scripted-random driver so every collection run is bit-exactly reproducible.

Why this matters. Making the paper a deterministic function of the release artifact ensures that released data, analysis scripts, and paper figures remain tightly coupled. The Python cache simulator used in Section 6 is included in the public artifact release.

8 Discussion

Implications for CDN operators. Our findings suggest that agentic workloads require different cache provisioning strategies than human traffic. The 69% rendering overhead means that agents waste the majority of CDN bandwidth on content they do not consume, a cost borne by both the publisher (origin load) and the infrastructure provider (cache capacity). The cache policy sensitivity implies that CDN operators cannot simply deploy a single eviction policy for all edge caches: smaller caches at the network edge benefit from size-aware policies (GDSF), whereas larger aggregation caches benefit from recency-based policies (LRU). This contrasts with production CDN traces [6, 38] where LRU-family policies typically dominate because human browsing produces more uniform object size distributions. Workload-aware policy selection [46] becomes critical as agent traffic grows.

Who benefits from these traces. Cache researchers get a real-trace replacement for synthetic AI-traffic overlays [46]; *agent framework developers* get a rendering-cost baseline (97.1% non-HTML) to evaluate structured-endpoint alternatives (e.g., x402 [41]); *publishers* can move beyond the block-vs-serve binary with per-request behavior.

Agent vs. human cache behavior. To check whether the GDSF advantage is specific to agentic workloads or a generic property of the underlying trace structure, we constructed a *synthetic* human browsing trace parameterized from HTTP Archive aggregate statistics (500 simulated page loads, 72 requests per page, 45% images, 33% JavaScript, 30% shared CDN resources) and replayed it through the same six policies. On the synthetic human trace, frequency-adaptive policies (S3-FIFO) dominate at all sizes (20.2% at 10 MiB vs. LRU 3.7%), whereas on the agent trace GDSF leads (66.4% at 10 MiB). We emphasize this comparison is directional only: the human trace is synthetic and does not capture real human session structure or temporal correlation. Replaying real public traces (e.g., HTTP Archive HAR data) is a priority extension. The qualitative takeaway (that agentic workloads favor different policies than human-shaped workloads) is consistent with [46]'s synthetic overlay study but warrants real-trace validation.

Rendering overhead. For every byte of HTML content, agents download 34× more total bytes (rendering infrastructure accounts for 97.1% of transferred volume vs. 2.9% HTML). This overhead motivates proposals for structured machine-readable endpoints and cryptographic agent identity schemes (e.g., x402 [41]). Whether such endpoints are practical at scale remains an open question; BrowseTrace provides the measurement substrate to evaluate their potential infrastructure impact.

Human traffic comparison. The HTTP Archive [19] reports human browsing at ∼72 requests per page with images at ∼45% of bytes and JavaScript ∼33%. Scripted Zurich sessions in BrowseTrace average 148 requests per multi-page session with JavaScript at 62.1% and images at 13.2%, an *inversion in content mix* (the per-session vs. per-page comparison is not directly volume-comparable) reflecting that agents trigger the rendering pipeline but do not request viewport images. This complements Zhang et al. [46]'s synthetic overlay finding with real browser-captured traces, additionally identifying the size-dependent GDSF advantage.

Scope as infrastructure benchmark. Unlike agent-evaluation benchmarks that rank task-completion performance, BrowseTrace measures network-level effects of agent behavior; leaderboard metrics are orthogonal. The release protocol is modeled on WebArena [48] and SWE-bench but publishes request-level traces rather than task specifications, enabling downstream systems research that requires realistic request streams.

9 Limitations and Next Steps

LLM backend diversity. The expanded collection covers six models from five providers, including frontier closed-source (GPT-4.1-mini, Gemini Flash/Pro, Claude Haiku 4.5), an agent-purpose-trained model (DeepSeek-V3.2 [12]), and an open-weight edge model (Qwen 2.5-Coder 7B [20]). Multi-region replication across three GCP regions confirms geographic invariance ($p > 0.45$ for all models). Human-driven traces remain absent; pairing with real user sessions is future work.

Collection substrate heterogeneity. Zurich sessions use BrowserUse on macOS while cloud sessions use Playwright on Linux in Docker. The raw 19× Zurich-to-cloud ratio conflates browser configuration with geographic variation; cloud-only comparisons on the same Docker image show 1.0–1.4× across regions ($p > 0.45$). Future releases will use a single containerized substrate across all regions.

CDP instrumentation scope. The CDP tracer captures requests for the active browsing context; requests during domain transitions or within opaque cross-origin iframes may be missed. HAR cross-referencing on 20 sessions bounds this undercounting below 3% of total requests.

Task completion proxy. A *reached-content* heuristic (≥3 domains, ≥5 status-200 responses) indicates 89.2% of scripted and 91.3% of LLM sessions reached meaningful content; semantic correctness labels are orthogonal to this network-level contribution and are planned as a future extension.

Scope. The 10 task families cover read-only navigation; interactive workflows (forms, authentication) are excluded

for ethical reasons. Repeated collection is needed for longitudinal stability; HTTP/2 multiplexing and HTTP/3 effects are recorded but not analyzed.

10 Conclusion

Browser-mediated AI agents exercise the web stack differently from human and scripted traffic, yet the community has lacked request-level evidence to study them. The first `browseruse-live-v3` release (1,301 sessions, six models, five providers) documents 2–5× request amplification, a rendering-dominated byte mix (97% non-HTML), and a cache-policy reversal on agent traffic. Infrastructure tuned on human traces will underperform on agentic workloads. More importantly, the release gives the community a common replay surface for future human-vs-agent comparisons, workload evolution studies, and workload-aware systems design.

References

[1] Anthropic. 2025. Claude Haiku 4.5. https://www.anthropic.com/news/claude-haiku-4-5.

[2] Martin Arlitt and Tai Jin. 2000. A Workload Characterization Study of the 1998 World Cup Web Site. *IEEE Network* 14, 3 (2000), 30–37. doi:10.1109/65.844498

[3] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. 2018. LHD: Improving Cache Hit Rate by Maximizing Hit Density. In *USENIX NSDI*.

[4] László A. Belady. 1966. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal* 5, 2 (1966), 78–101. doi:10.1147/sj.52.0078

[5] Daniel S. Berger, Benjamin Berg, Timothy Zhu, Siddhartha Sen, and Mor Harchol-Balter. 2018. Practical Bounds on Optimal Caching with Variable Object Sizes. In *Proceedings of the ACM on Measurement and Analysis of Computing Systems (SIGMETRICS)*. doi:10.1145/3179405

[6] Daniel S. Berger, Ramesh K. Sitaraman, and Mor Harchol-Balter. 2017. AdaptSize: Orchestrating the Hot Object Memory Cache in a Content Delivery Network. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[7] Oleksii Borysenko. 2026. Developer Experience with AI Coding Agents: HTTP Behavioral Signatures in Documentation Portals. arXiv preprint arXiv:2604.02544.

[8] Browser Use. 2024. BrowserUse: Open-Source Browser Automation for AI Agents. https://github.com/browser-use/browser-use.

[9] Michael Butkiewicz, Harsha V. Madhyastha, and Vyas Sekar. 2011. Understanding Website Complexity: Measurements, Metrics, and Implications. In *Proceedings of the ACM Internet Measurement Conference (IMC)*. doi:10.1145/2068816.2068846

[10] Ludmila Cherkasova. 1998. *Improving WWW Proxies Performance with Greedy-Dual-Size-Frequency Caching Policy*. Technical Report HPL-98-69(R.1). Hewlett-Packard Laboratories.

[11] Cloudflare. 2024. Radar Year in Review 2024 — Traffic and Bot Trends. Cloudflare Radar, https://radar.cloudflare.com/year-in-review/2024. Accessed April 2026.

[12] DeepSeek-AI. 2026. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. https://huggingface.co/deepseek-ai/DeepSeek-V3.2. Reasoning-first model with DeepSeek Sparse Attention (DSA) and large-scale agentic post-training over 1,800+ interactive environments..

[13] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *NeurIPS Datasets and Benchmarks Track*.

[14] Gil Einziger, Ohad Eytan, Roy Friedman, and Ben Manes. 2022. Lightweight Robust Size Aware Cache Management. *ACM Transactions on Storage* 18, 3 (2022). doi:10.1145/3507920

[15] Gil Einziger, Roy Friedman, and Ben Manes. 2017. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Transactions on Storage* 13, 4 (2017). doi:10.1145/3149371

[16] Phillipa Gill, Martin Arlitt, Zongpeng Li, and Anirban Mahanti. 2007. Youtube Traffic Characterization: A View from the Edge. In *Proceedings of the ACM Internet Measurement Conference (IMC)*. doi:10.1145/1298306.1298310

[17] Fabian Hantke et al. 2025. Reproducible, Accurate, and Archivable Web Measurements. In *USENIX Security Symposium*.

[18] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

[19] HTTP Archive. 2025. State of the Web. https://httparchive.org/. Accessed March 2026.

[20] Binyuan Hui, Jian Yang, et al. 2024. Qwen2.5-Coder Technical Report. https://arxiv.org/abs/2409.12186.

[21] Sunghwan Ihm and Vivek S. Pai. 2011. Towards Understanding Modern Web Traffic. In *Proceedings of the ACM Internet Measurement Conference (IMC)*. doi:10.1145/2068816.2068845

[22] Christos Iliou, Theodoros Kostoulas, Theodora Tsikrika, Vasilios Katos, Stefanos Vrochidis, and Ioannis Kompatsiaris. 2019. Towards a Framework for Detecting Advanced Web Bots. In *Proceedings of the 14th International Conference on Availability, Reliability and Security (ARES)*. doi:10.1145/3339252.3339267

[23] Imperva/Thales. 2024. Bad Bot Report 2024. https://www.imperva.com/resources/resource-library/reports/bad-bot-report/.

[24] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? (2024).

[25] Taein Kim, Karstan Bock, Claire Luo, Amanda Liswood, Chloe Poroslay, and Emily Wenger. 2025. Scrapers Selectively Respect robots.txt Directives: Evidence From a Large-Scale Empirical Study. In *ACM Internet Measurement Conference (IMC)*.

[26] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. 2024. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

[27] M. Koster, G. Illyes, H. Zeller, and L. Sassman. 2022. *Robots Exclusion Protocol*. Technical Report RFC 9309. IETF. doi:10.17487/RFC9309

[28] Juhi Kulshrestha, Marcos Oliveira, Orkut Karacalik, Denis Bonnay, and Claudia Wagner. 2021. Web Routineness and Limits of Predictability. *Proceedings of the International AAAI Conference on Web and Social Media* (2021).

[29] Enze Liu, Elisa Luo, Shawn Shan, Geoffrey M. Voelker, Ben Y. Zhao, and Stefan Savage. 2025. Somesite I Used To Crawl: Awareness, Agency and Efficacy in Protecting Content Creators From AI Crawlers. In *ACM Internet Measurement Conference (IMC)*.

[30] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[31] Carlos López, Daniel Morató, Eduardo Magaña, and Mikel Izal. 2022. Validation of HTTP Response Time From Network Traffic. *IEEE Transactions on Network and Service Management* (2022). doi:10.1109/TNSM.2022.3181708

[32] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. (2024).

[33] Nimrod Megiddo and Dharmendra S. Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *Proceedings of the USENIX Conference on File and Storage Technologies (FAST).*

[34] Microsoft. 2024. Playwright: Fast and Reliable End-to-End Testing. https://playwright.dev/.

[35] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. 2021. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332* (2021).

[36] Kevin Ruth, Deepak Kumar, Brandon Wang, Luke Valenta, and Zakir Durumeric. 2022. Toppling Top Lists: Evaluating the Accuracy of Popular Website Lists. In *Proceedings of the 22nd ACM Internet Measurement Conference (IMC).* doi:10.1145/3517745.3561444

[37] Abdur-Rahman Ibrahim Sayyid-Ali, Daanish Uddin Khan, and Naveed Bhatti. 2026. Are LLM Web Search Engines Sustainable? A Web-Measurement Study of Real-Time Fetching. In *The Web Conference.*

[38] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. 2020. Learning Relaxed Belady for Content Distribution Network Caching. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*

[39] TollBit. 2024. TollBit: Monetize AI Access to Your Content. https://tollbit.com/.

[40] Carl Waldspurger, Trausti Saemundsson, Irfan Ahmad, and Nohhyun Park. 2017. Cache Modeling and Optimization using Miniature Simulations. In *Proceedings of the USENIX Annual Technical Conference (ATC).*

[41] x402 Foundation. 2025. x402: HTTP Payment Protocol. Coinbase and Cloudflare.

[42] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems (NeurIPS).*

[43] Juncheng Yang and contributors. 2024. libCacheSim: A high-performance library for cache simulations. https://github.com/1a1a11a/libCacheSim.

[44] Juncheng Yang, Ziyue Qiu, Yazhuo Zhang, Yao Yue, and Rashmi Vinayak. 2023. GL-Cache: Group-Level Learning for Efficient and High-Performance Caching. In *Proceedings of the USENIX Conference on File and Storage Technologies (FAST).*

[45] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and Rashmi Vinayak. 2023. FIFO Can Be Better than LRU: The Power of Lazy Promotion and Quick Demotion. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP).* doi:10.1145/3600006.3613147

[46] Yazhuo Zhang, Jinqing Cai, Avani Wildani, and Ana Klimovic. 2025. Rethinking Web Cache Design for the AI Era. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC).* doi:10.1145/3772052.3772255

[47] Yazhuo Zhang, Juncheng Yang, Yao Yue, Ymir Vigfusson, and K. V. Rashmi. 2024. SIEVE is Simpler than LRU: an Efficient Turn-Key Eviction Algorithm for Web Caches. In *USENIX NSDI.*

[48] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *Proceedings of the International Conference on Learning Representations (ICLR).*

A Human Browsing Baseline

To our knowledge, no public dataset provides human browsing traces at HTTP request granularity for task-based sessions. Existing datasets are either automated crawls (HTTP Archive), URL-visit level (Zenodo 2,148-user dataset [28]), controlled lab studies (IEEE DataPort [31]), or unreleased private collections. For illustrative comparison, a single human self-study session across all 10 tasks was collected using identical CDP instrumentation; no external participants were recruited, and local policy did not require human-subjects review (see Ethics). The session is limited by sample size (N=1) and is presented as a qualitative reference point only, not as a human baseline. The participant generated a mean of 529 requests per task (range: 15–1,929), compared to the scripted-random mean of 148 and the LLM-agent range of 197–562. The human tracer captures requests from the primary browsing context; requests initiated in manually opened new tabs are not captured, which may undercount total human traffic. Additionally, human sessions use stock Chromium UA headers while agent sessions declare a project-specific benchmarking UA, which may trigger different server-side responses such as anti-bot challenges. Full session traces are included in the public artifact release.

B Ethics

All sessions target publicly accessible web pages. The collector does not authenticate, submit forms, execute payments, or interact with login flows. Each run is limited to a small navigation budget on configured target domains, with rate-limiting sleeps between page visits to avoid bursty load on target servers. The released artifact contains request-level metadata and response sizes but no response bodies or personally identifiable information. Before release, we strip `Cookie`, `Authorization`, `Set-Cookie`, and `Proxy-Authorization` request/response headers, and we sanitize URL query strings by retaining only parameter names (values replaced with `_REDACTED_`). This uniform replacement preserves URL path uniqueness for cache keying while removing potentially identifying parameters. The collector never logged in to any target site, so session tokens cannot appear in the trace by construction. We reviewed each target domain's published robots.txt and limited collection to publicly accessible landing and listing pages; where a specific path was disallowed for general user-agents, we either avoided it or restricted collection to the minimal subset needed for characterization. We declared a benchmarking user-agent and rate-limited all sessions. The benchmark collection infrastructure does not involve human subjects. The qualitative

Table 6: Per-region cache hit rates (replaying each region's trace in isolation). GDSF's advantage at small-to-mid cache sizes is consistent across the three cloud regions; Zurich (narrower content mix) favors LRU at 10 MiB and up. Boldface marks the winning policy per (region, size) cell.

	5 MiB		10 MiB		50 MiB	
Region	LRU	GDSF	LRU	GDSF	LRU	GDSF
Zurich	0.486	**0.704**	**0.813**	0.768	0.819	0.819
US Central	0.334	**0.532**	0.407	**0.585**	0.670	0.664
EU West	0.383	**0.591**	0.504	**0.650**	0.731	0.727
Asia Southeast	0.343	**0.562**	0.436	**0.618**	0.693	0.687

self-study session in Appendix A involved no external participants; under local institutional policy, such self-study sessions fall outside human-subjects review requirements. URL-level traces could theoretically enable site-level traffic profiling when combined with external data, but query-string redaction and the exclusion of response bodies mitigate this risk. Each session visits fewer than 20 pages per site; per-region, per-task loads are modest (fewer than 2,000 requests per task per region).

Use of generative AI.. Portions of the analysis pipeline code and manuscript text were drafted with the assistance of large language models: Anthropic Claude Opus 4 and OpenAI GPT-4.1-mini. Specifically, the statistical analysis scripts (`rigor_sweep.py`, `round9_analysis.py`, `llm_cache_replay.py`) were co-developed with LLM assistance, and several expository paragraphs were drafted with LLM suggestions that were subsequently reviewed, revised, and verified by the authors. No portion of the paper was submitted to third-party AI tools during the review period. Per ACM policy, we disclose this use and confirm that all authors take full responsibility for the content.

Fingerprint disclosure. The browser fingerprint is stock Playwright/BrowserUse Chromium with default settings and a project-specific benchmarking User-Agent suffix (redacted for review).

IRB status and disclosure. Our IRB policy does not require review for measurement of publicly accessible web resources without human subjects. We treat any takedown request as binding and will share our methodology on request.

Blocking / throttling events. Sporadic anti-bot blocking affected <1% of sessions; these are flagged in the per-session `navigation_status` field.

Table 7: Top-3 most influential cells (Cook's D). Thresholds: Cook's $D > 4/n = 0.133$; $|\mathbf{DFFITS}| > 2\sqrt{2/n} = 0.516$. Removing the top cell leaves the power-law intact ($R^2 = 0.62$).

Task	**Model**	V_s	amp	Cook's D	LOO R^2
real-estate	Pro	160	3.2	0.48	0.62
literature-review	Flash	240	25.7	0.15	0.48
regulatory-lookup	GPT	390	21.5	0.11	0.51

C Per-region Cache Hit Rates

D Regression Robustness

We report full leave-one-out (LOO) and influence diagnostics for the power-law fit $amp = a \cdot V_s^b$ (Section 5.8). OLS on the 30 (model, task) cells: $b = -0.59$, standard error 0.11, bootstrap 95% CI $[-0.82, -0.36]$ ($B = 10{,}000$), $R^2 = 0.51$, Kendall's $\tau = -0.47$ ($p < 0.001$), Spearman's $\rho = -0.64$ ($p < 0.001$). The fit on per-task means ($n = 10$) yields $R^2 = 0.72$ and $b = -0.62$.

Leave-one-out sensitivity. Across the 30 LOO refits, R^2 stays in $[0.48, 0.62]$ and exponent b in $[-0.69, -0.54]$ (std $= 0.024$). The power-law structure is not driven by any single cell.

Influence diagnostics. Residuals are consistent with normality (Shapiro-Wilk $W = 0.98$, $p = 0.90$; Anderson-Darling $A^2 = 0.16$, below the 15% critical value 0.52). All diagnostics are reproducible via `analysis/loo_influence.py` in the released artifact.